

A simple system for local astrophotography data organization

An organizer and tracker that derives processing state directly from the files.

A dashboard to see all your captured data — target lists with integration time and the current image-processing state of every target.

A few scripts using controlled vocabularies, a session-keyed folder structure, and consistent calibration make keeping your library a simple task.

Steve Stanzel · July 2026 · revision 2, for peer review

425.444.3552 · sstanzel@hotmail.com

Abstract

This paper describes the system I use to organize an astrophotography dataset that currently spans 216 deep-sky imaging sessions and 753 hours of kept integration on 72 imaged targets, captured with four telescopes and four cameras over three seasons. The goal is a library that is legible — to me, to my processing tools, and to anyone reviewing the work — without a heavyweight digital asset manager and without any bookkeeping that has to be maintained by hand.

Two ideas carry the whole design. First, setup is deliberately light: the entire management layer is one configuration file naming the capture libraries, plus a small set of plain Python scripts run from the command line. Second, and more important, every piece of state the system reports is derived automatically from the files on disk. Whether a session has been culled, integrated, or published; which stacking software produced a master; which calibration masters exist, are stale, or are still owed — none of it is typed into a spreadsheet. The scripts read the folder names, the capture filenames, and the presence or absence of specific files, and the answers fall out. If the files are right, the reports are right; there is no second copy of the truth to drift out of date.

The system has four parts. A controlled-vocabulary registry of empty directories defines the only legal names for filters, telescopes, cameras, and target objects. A Post Haste template creates an identical folder structure for every capture night, so the processing tools always find what they expect. A shared calibration library holds bias and dark-frame sets organized by camera and settings, reusable across every target. And a SQLite-backed tracker reads the ASIAir and

NINA filename grammars to answer the questions a serious imager asks: how many hours on a target this year and lifetime, which telescope-camera combinations have covered it, where each session sits in the processing pipeline, what to work on next, and which calibration data still needs to be captured or turned into masters.

The paper documents the conventions, the data structures, and the toolchain, and closes with appendices for readers who want to adopt the system against an existing library. The code will be shared as a public repository so the scripts can be run, not just read.

The outcome, first

Before the conventions, here is what they buy. After any capture night, one command — `refresh.py` — rescans both capture libraries (roughly 15,000 frames, on the order of a minute), re-derives every state in the system from the files themselves, regenerates the dashboard and the Excel workbook, and copies both to a cloud-synced folder so they stay readable even when the capture volumes are dismounted. Nothing else is maintained: no spreadsheet updated by hand, no checklist ticked. The dashboard that command produces opens like this:

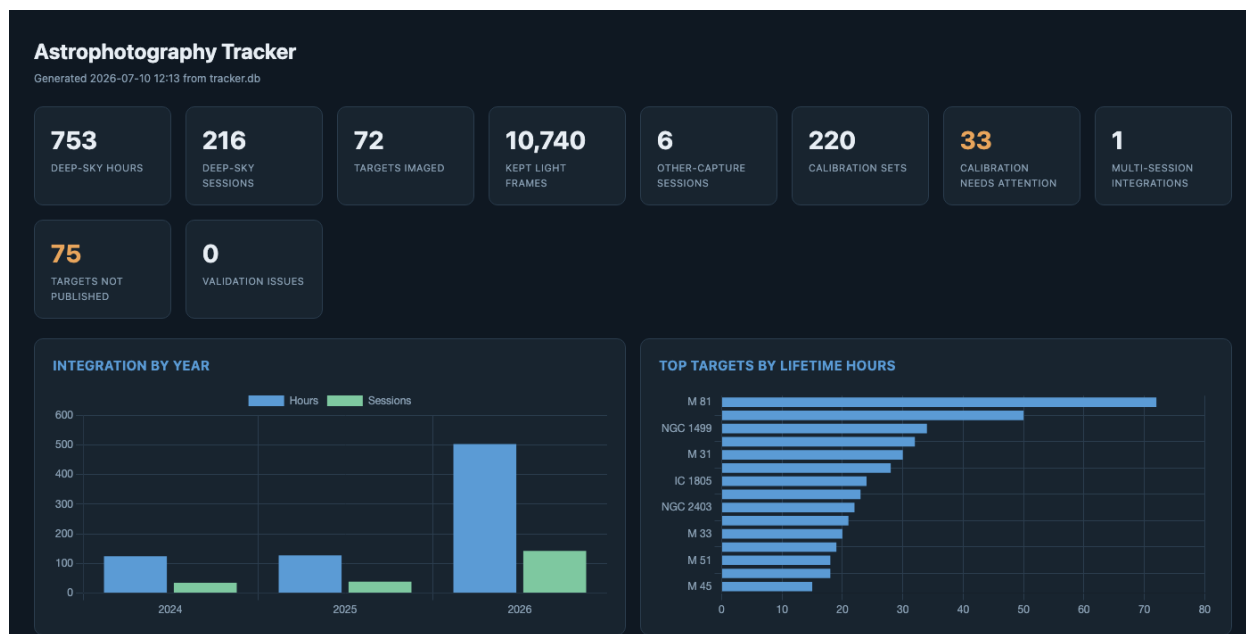


Figure 1 — the dashboard's summary layer, regenerated by one command after each capture night: headline cards, integration by year, and top targets by lifetime hours.

Below that summary layer sit the pipeline tables, the work queue of what to do next, per-target and per-session detail, and the calibration and data-health panels — all of it derived. The rest of this paper explains the small set of conventions that make that derivation possible; keep this picture in mind as the destination they all serve.

1. Why a written convention

Astrophotography data degrades quietly. The night you captured M 81 is unambiguous in the moment, but a year later — with more sessions piled on top of it from several telescope-camera combinations — you can be reduced to opening FITS headers to remember what was what. The convention here is the smallest set of rules that, applied consistently, lets a script (or a future version of yourself) answer questions about the capture tree without opening a single image.

A few questions motivated the design:

- How many hours of usable integration do I have on a given target, this year and lifetime?

- Which telescope-camera combinations have imaged a given target, and which still need data collected?
- Where in the processing pipeline is each session right now, and what should I work on next?
- Does the data I have actually captured have matching calibration — the right bias and dark masters for every gain, exposure, and temperature my light frames use?

If the folder tree cannot answer these questions based on the structure, then the small text-file sidecar and a local HTML dashboard should be able to.

2. Goals and principles

Six principles shape this design:

- **Self-describing folders.** Every directory name tells a parser the target, the rig, and the night without opening anything inside it.
- **Controlled vocabularies.** Filter, telescope, camera, and target labels are drawn from a fixed registry of empty directories. The list of valid values is itself a set of folders, so the filesystem is the authority. If you get a new sensor, just add a new folder to add it — and that name will be used consistently throughout your libraries.
- **One folder structure per night.** Post Haste (Digital Rebellion's free folder-templating application) creates the same session folder structure every time, so PixInsight and PIMagic Studio always find what they need.
- **Filename metadata, not sidecar metadata.** ASIAir (ZWO's capture controller) encodes exposure, gain, camera, binning, datetime, and sensor temperature in every frame's filename; NINA (the Windows capture suite) encodes the same plus per-frame focus and guiding quality. The system reads it and doesn't recreate it.
- **Derived state, never recorded state.** A session is “culled” because frames sit in its Rejected folder; “integrated” because its Results folder holds output; a calibration set “has a master” because a master file sits inside it. The tracker never asks you to declare these things — it observes them, every scan, from scratch. The single deliberate exception is the “edited” flag in the per-session and per-integration text file, because no file reliably distinguishes a finished edit from an unedited master.
- **Calibration organized by reusability.** Bias and dark frames depend only on the camera and its settings, so they live once in a shared library and are pre-built into masters. Flats and dark-flats depend on the night's focuser position and dust, so they stay with their session.

Together these make the dataset structurally queryable — integration totals come from filenames, pipeline state comes from folder contents, and no FITS-header parsing is required for the common case.

3. The two capture libraries and the registry

Captures live on two physical volumes that share an identical internal structure:

- **SSD — the working volume.** Current-year imaging on a fast external solid-state drive. New sessions land here and most active processing happens here.
- **NAS — the lifetime archive.** Prior-year sessions on a network-attached storage volume. Sessions migrate from the working volume to the archive after a year is closed out.

Every directly visible directory at a library root is a target — one folder per object, named with the catalog identifier and common name (for example, “M 81 Bodes Galaxy”). The exceptions are the shared calibration library (Section 6), a folder for live-stacking output, and the “other captures” buckets (Section 7). Inside each target folder the children are session folders, one per night per rig, plus an optional integrations folder (Section 9).

The tracker is configured with the list of libraries in a single TOML file, and scans all of them; a query for lifetime hours on a target aggregates across volumes transparently. Adding a third library — a second archive, a travel drive — is one more block in the configuration file.

3.1 The registry: `_organization/`

The registry is the vocabulary authority, and it is deliberately empty: each subdirectory under `_organization/` is a vocabulary, and the directory's contents are the legal values. The names are the data. It lives with the management scripts, outside the capture libraries, so there is exactly one authority no matter how many libraries exist.

Vocabulary	Members	Purpose
<code>scope_values/</code>	9	Every telescope and lens used for any kind of astronomical imaging.
<code>sensor_values/</code>	10	Every camera. “Sensor” is the term most astronomy tooling uses, although it names the whole camera, not just the chip.
<code>scope+sensor_values/</code>	17	The telescope-and-camera combinations that pair for deep-sky work. Only these may appear in session folder names.
<code>target folders/</code>	81	The catalog identifier and common name of every object imaged or planned. The folder is created in the registry first; a copy appears in a capture library the first night the object is imaged.
<code>filter_values/</code>	14	Every filter used on any rig, each named with a short label and a descriptive suffix.

Because the registry is folders, adding a camera is one `mkdir` — and every validation and report immediately knows the new name is legal. The full vocabulary lists are reproduced in Appendix A. The tracker validates in both directions: a session folder using a name that is not in the

registry is flagged, and so is a calibration folder (Section 6), so a typo cannot quietly create a phantom camera.

4. The per-session structure

Each imaging night is organized into the same set of folders and files created by Post Haste from a template, with the following structure:

```
{Target_id} {Scope} {Sensor} {YYYY-MM-DD}/
├── Light/                               ← raw light frames
├── Rejected/                             ← culled (rejected) lights
├── Flat {Scope}_{Sensor} {Date}/
│   ├── Flat/                             ← that night's flats
│   └── Dark Flat/                         ← matching dark-flats
├── PI Process/                           ← PixInsight intermediates
├── (scratch)
├── PI Magic/                             ← PIMagic Studio runs (scratch)
├── {session_name}.pxiproject/            ← PixInsight project
├── {session_name} Results/               ← the keepers (master + exports)
└── {session_name} notes.toml            ← per-session metadata (Section
4.1)
```

The folders divide into three roles, and the roles carry the data-retention policy. Raw data — Light, Rejected, Flat and Dark Flat files — is not modified by any process. The Results folder holds the output of processing you want to keep: the integrated master and any exports or edit files, the things other programs need to open. PI Process and PI Magic are scratch folders recreatable from the raw data — intermediates that a cleanup script empties to reclaim space. One rule ties the roles together: the keepers must reach Results before the scratch folders are swept. A promotion script copies masters and edit files from the working folders into Results, and the cleanup script refuses to empty any session whose master has not been promoted — so a cleanup can never lose a finished image.

Two more choices are opinionated by design. In-session calibration is limited to flats and dark-flats, which depend on the night (Section 6 covers the rest). And every session has its own PixInsight project — a project is tied to a single night, and combining across nights is a first-class concept of its own (Section 9).

4.1 The notes file: what only the night can tell you

Every session folder carries a notes file — {session name} notes.toml — created with the folder structure. Its purpose is narrow and important: to record the things neither the folder name nor the frame filenames can carry. The session's identity (target, telescope, camera, date) is deliberately absent — it is already in the folder name and in every frame — so the file holds only conditions, observations, and downstream status. The format is TOML: comment-friendly, diff-friendly, and dependably parseable. The tracker reads every notes file on every scan, so a note edited on any machine reaches the next report with no synchronization step.

Most of the file fills itself. At ingest, the tracker computes the moon section from the session date and the site coordinates, and pulls the weather section — temperature, humidity, dew point, cloud cover, wind, pressure — once from a free historical-weather service for that date and location; a value already present is never overwritten. What remains for the human is exactly what should be: the site key (once), free-text observations from the night, processing commentary, the one manual pipeline flag (edited), the publication and print records of Section 10, and reprocessing to-dos the tracker aggregates into one queryable list.

Section	Filled by	Contents
location	hand, once	A key into a small locations file (site coordinates, Bortle sky-brightness class).
[sky]	auto	Moon phase, illumination percentage, and age in days — computed from the session date and the site coordinates at ingest.
[weather]	auto	Temperature, humidity, dew point, cloud cover, wind, and pressure — pulled once from a historical weather service for the session's date and location. Seeing and transparency remain hand-filled.
[observation]	hand	Free-text notes from the night — equipment quirks, passing cloud, satellite trails. The genuinely irreplaceable field.
[processing]	hand	Commentary on processing attempts — what worked, what to redo.
[integration]	hand (one flag)	edited = true, the pipeline's one manual flag (Section 8).
[[published]] / [[printed]]	hand, per event	The ledger blocks of Section 10.
[future_processing]	hand	Short to-do lines; the tracker aggregates them across all sessions into one queryable processing to-do list.

Because these conditions land in the database next to the frames, questions that would once have meant paging through a logbook become trivial: what have I captured within a day of a new moon; how many sessions ran below freezing; what does my integration time look like by month. None of those queries is written into the tracker's views yet — but every field they need is already in the database, so each is one SELECT away, and the dashboard can grow whichever of them earn a permanent place.

5. Naming conventions

5.1 Target folder name

A target folder's name is the catalog designation, a space, and the common name:

```
M 81 Bodes Galaxy
NGC 1499 California Nebula
SH2 277 Flame Nebula
```

All catalogs except Sharpless use a single space between catalog and number (M ##, NGC ####, IC ####, C ##, LDN ####, HR ####). Sharpless objects use the compact form SH2 ### — “SH2” as one token — which matches the Sh2-### form used in the literature and keeps the name to the standard catalog-space-number shape. The common name is Title Case for a proper noun (“California Nebula”) and lower case for a descriptive label (“spiral galaxy”).

When a single field contains more than one named object — the Leo Triplet, NGC 3718/3729, Markarian's Chain — the folder is named after the field, not split into per-object folders. Companion objects are recorded in the tracker's metadata rather than the filesystem, which avoids dummy session folders and keeps per-night integration totals unambiguous.

5.2 Session folder name

A session folder is one target, one acquisition night with one telescope and one camera. The name is four whitespace-delimited tokens — the target identifier with underscores joining its parts, the telescope, the camera, and the local calendar date the session started:

```
M_81 RASA8 ASI2600MCAir 2026-04-19
NGC_1499 Pleiades111 ASI2600MCAir 2026-02-26
```

The tuple (target, telescope, camera, date) is the session's natural key; across the full dataset it is unique with no collisions, which is what lets the tracker re-scan endlessly without duplicating anything. One suffix extends the grammar: when a second rig rides along pointed at a field next to the main target, the session carries an _adjacent suffix on the target token (for example M_12_adjacent Redcat51 minicam8 2026-07-06) and files inside the base target's folder, so the piggyback data is recorded without inventing a fake target.

5.3 Light-frame filename grammars

Two capture systems write subtly different filenames, and both have changed format over time. The tracker's parser recognizes nine grammars — ASIAir science and calibration frames, their DSLR variants (where ISO stands in for gain), an early-2026 ASIAir firmware variant that moved the rotator angle, and NINA's current and legacy patterns. Representative examples:

Grammar	Example
ASIAir science	Light_NGC 3718_120.0s_Bin1_585MC_gain252_20260424-011157_74deg_-18.7C_LQuadE_0054.fit
ASIAir calibration	Flat_108.3ms_Bin1_585MC_gain200_20260309-084517_156deg_-19.5C_0008.fit

Grammar	Example
NINA science	LIGHT_M 106_300.00s_Bin1x1_Poseidon-C PRO_gain125_2026-05-12_22-56-19_288.99deg_-20.00C__HFR2.83_RMS0.42_LQuadE_0001.fits
ASIAir DSLR	Light_Orion_300.0s_Bin1_ISO1600_20240406-221530_23.0C_R5_0012.fit

In every grammar each field is structurally extractable: target (when present), exposure, binning, camera, gain, datetime, optional rotation angle, sensor temperature, optional filter, and a sequence counter. Because the data is in the filename, totalling integration on a target reduces to walking the light frames, parsing each name, and summing — only frames rejected during culling and sub-second preview frames are excluded. The NINA science grammar additionally carries HFR (half-flux radius, a focus-quality measure) and RMS (guiding error in arcseconds), which lets the tracker flag poor frames from filenames alone. The full reference patterns are in Appendix C.

6. Calibration, organized by reusability

Calibration frames divide cleanly by what they depend on, and that dependency decides where they live:

- **Flats and dark-flats depend on the night** — the focuser position, the camera's rotational alignment, and the dust state of the optical train. They stay with the session they calibrate, in a subdirectory named for the rig and date. When several targets share one flat set on a multi-target night, the other sessions reference the set in their notes file rather than duplicating it.
- **Bias and dark frames depend only on the camera and its settings** — gain for bias; gain, exposure, and sensor temperature for darks. They are captured once per combination, live in a shared calibration library, and are worth pre-building into master frames that are then reused for six months to a year.

The shared library currently holds 133 bias and dark sets, organized around hardware rather than targets:

```

_Calibration Library/
├─ Bias/{Camera}/Bias {Date}/          ← dated bias sets
├─ Dark/{Camera}/{Temp}/{Gain}/{Exposure}/Dark {Date}/ ← dated dark sets
└─ !camera/ ...                        ← duplicate-and-
rename templates

```

Two conventions make the library machine-readable. First, camera folder names are drawn from the same `sensor_values` registry as session names, so a light frame's camera and a dark set's camera match by exact string — the key that makes the coverage report in Section 11 possible. Second, “this set has a master” is detected from the files themselves: when a master frame (`master*.xisf` or equivalent) is dropped into a set's folder, the tracker sees it on the next scan and the set counts as mastered. No list of masters is maintained anywhere. The parser

also reads set parameters from either the folder tree ({Temp}/{Gain}/{Exposure}) or from ASIAir-style set names such as Dark_300.0s_Bin1_2600MC_gain100_-20.0C, so bulk dark libraries captured in one sitting need no re-folding.

Each calibration class carries thresholds — a minimum frame count and a refresh age in days — stored in a configuration file alongside the acquisition goals, not in code. Every scan classifies each bias and dark combination as ok, no master (raw frames waiting to be built), or stale (new raw frames captured after the master was built, or the master older than its refresh age). The classification feeds the work queue directly.

7. “Other captures” — the non-conforming category

Not every astronomy session goes through the standard imaging rigs. Lunar and planetary imaging, comets and asteroids, nightscapes and time-lapse work with DSLR and mirrorless bodies, science programs, and one-off test shoots share a property: they do not fit the four-token session grammar, and their frame counts would distort deep-sky integration statistics. They live in a set of top-level buckets that the tracker records but deliberately exempts from the naming conventions and excludes from deep-sky totals: Asteroids Comets, Moon, Planets, Nightscapes (which includes time-lapse work), a miscellaneous bucket, and five science buckets — Astrometry, Spectroscopy, Photometry, Exoplanets, and Double Stars. The science topics could have nested under one Science parent, but a second hierarchy level does not keep things simple; every bucket lives at the top, one level deep, alongside the targets. Named-star sessions of single bright stars are not “other captures”: they use a standard imaging rig and follow the normal grammar.

8. The processing pipeline: seven stages, derived from files

Every image moves through the same seven stages. What makes the ladder useful is that each stage — with one deliberate exception — is derived automatically from the files, so the pipeline view is always current the moment the scan runs:

#	Stage	How the tracker knows
0	Planned	A target folder exists in the registry but no session has been captured.
1	Captured	A session folder with light frames exists.
2	Culled	Frames sit in the session's Rejected folder — or the session has already been integrated (the stacking tools cull as they go). A reviewed night where every frame was kept can be marked with one line in its notes file; this is the rare case.
3	Integrated	The session's Results folder holds output. Which software produced it — PixInsight or PIMagic Studio — is auto-detected from which

#	Stage	How the tracker knows
		working folder holds files, and the answer persists after the scratch folders are cleaned.
4	Edited	The one manual flag: edited = true in the session's notes file, because no file reliably distinguishes a finished edit from an unedited master.
5	Published	The notes file carries at least one publication record (Section 10).
6	Printed	The notes file carries at least one print record (Section 10).

Stages are tracked at two grains. A single night, viewed on its own, is a single-session integration: it has its own PixInsight project and Results folder and can be culled, integrated, edited, and published as a first-light image. A multi-session integration (next section) combines nights and moves through stages 3 to 6 itself. A target's position in the pipeline is simply the furthest any of its sessions or integrations has reached — so the pipeline can be read per target (“where does M 81 stand?”) or per session (“did this night's data ever reach a wall?”).

There is deliberately no “calibrate” stage and no record of which computer ran a stack. Calibration is applied inside the stacking tools at integration time (PixInsight's WeightedBatchPreprocessing and PIMagic Studio both fold the flats, dark-flats, and library masters in as they run), so it is not an observable state of the files; and machine tracking was tried and removed as bookkeeping without insight.

9. Multi-session integrations: living, rule-based

Combining nights is where most organizational schemes get vague — folders named “final v3 really final”. Here a multi-session integration is a folder under the target (`{target}/integrations/{target_id} {rig|composite} {span}/`) holding the stacking project, a Results folder for its master, and a small manifest file, `integration.toml`. The manifest has two halves, and the split is the point:

- **[membership] is a rule, resolved fresh every scan.** “All RASA8 + ASI2600MCAir sessions on this target, any year” is written once; as new nights are captured they match the rule automatically. This is the available data — nothing to maintain.
- **[built] is a record of what was actually stacked** into the current master — a list of session names, stamped by a helper script after each stack. This is the built data — truthful because it was recorded at the moment of stacking.

From the two halves the tracker derives available hours, built hours, the date the master's data runs through, and — most usefully — staleness: an integration whose rule now matches sessions that are not yet in its master is flagged for restacking, with the hours it is behind. One living folder exists per rig and span, restacked in place as data accumulates; versioned snapshot folders are reserved for frozen, published states. A composite integration (mixing rigs) is the same mechanism without the rig constraint.

10. Publishing and printing: a ledger, not a checkbox

Publishing and printing are records of events, not pipeline gates — a finished image can be posted several times in different venues and printed years apart. Each event is one block in the session's or integration's notes file:

```
[[published]]
kind  = "astrobin"          # astrobin | social | other
url   = "https://astrobin.com/.../"
title = "M 81 – 8.5 hours, RASA8"
date  = "2026-05-10"
```

```
[[printed]]
title = "16x20 metal print"
date  = "2026-06-02"
```

The tracker collects every block into a publications table and shows two ledgers — Published and Printed — with clickable links. The pipeline flags “published” and “printed” are then derived: an image is published if it has at least one publication record. Nothing is checked off by hand, which keeps the record and the state permanently consistent.

11. The tracker: data structure, initial ingest, and the three views

11.1 How the data is structured

The database is SQLite — a single file, no server — and its shape mirrors the folder convention directly:

```
libraries(library_id, root_path, role)          ← one row per configured
volume
targets(target_id, catalog, number, common_name, is_other_capture, ...)
sessions(session_id, target_id, scope, sensor, session_date, library_id,
         frame counts, integration_s, pipeline stages, notes metadata, ...)
         UNIQUE (target_id, scope, sensor, session_date)
frames(frame_id, session_id, frame_type, exp_s, gain, temp_c,
        captured_at_utc, filter, hfr, rms_arcsec, is_rejected, ...)
calibration_masters(class, camera, temperature_c, gain, exp_s,
                    frame_count, is_generated_master, ...) ← one row per set
integrations(...) + integration_members(...)    ← Section 9's two halves
publications(...)                             ← Section 10's ledger
target_goals(...) · calibration_thresholds(...) · processing_todos(...)
validation_findings(severity, code, message, ...) ← Appendix B
```

Session identity is the four-token natural key, enforced UNIQUE — the same tuple that names the folder. Frames belong to sessions; each frame row is one parsed filename. On top of the tables, a set of SQL views computes the derived answers — per-session pipeline stage, integration overview with staleness, calibration needs, and the light-versus-calibration coverage

report — so every reporting face reads the same logic. Representative queries are in Appendix D.

11.2 Initial ingest — from an existing library to a populated database

Setup is intentionally small. The management folder is cloned anywhere; the only machine-specific piece is one TOML configuration file:

```
[[library]]
id   = "working"
path = "/Volumes/fast-ssd/.../Astrophotography 2026"
role = "working"

[[library]]
id   = "archive"
path = "/Volumes/nas/.../Astrophotography Library"
role = "archive"
```

The first run of the ingest script creates the database from the schema and then does what every later run does: load the vocabularies from the registry; walk each library's target folders; parse every session folder name and every frame filename against the nine grammars; sum kept exposure into integration seconds; walk the calibration library into calibration sets; resolve integration manifests; read each session's notes file; and finish with a validation pass (Appendix B) that reports anything that did not parse or does not conform. The scan is idempotent — every row is upserted on its natural key, so re-running after a capture night refreshes the data without duplicating it, and all derived fields (frame counts, masters detected, stage flags) are recomputed from the current state of the disk. A full two-library scan of roughly 15,000 frames takes on the order of a minute.

Day to day, one command — `refresh.py` — chains the whole pipeline: ingest, then regenerate both report faces, then copy them to a cloud-synced mirror folder so the current dashboard is readable even when the capture volumes are dismounted.

11.3 The three views of the library

The database is the single source of truth; everything human-facing is generated from it and disposable:

- **The HTML dashboard** — a single self-contained file: headline cards, integration-by-year and top-target charts, the published/printed ledgers, pipeline tables, the work queue (next section), sortable and filterable tables for targets, sessions, and integrations, and the calibration and data-health panels.
- **The Excel workbook** — the same data as live SUMIF-driven sheets for anyone who wants to slice it by hand.
- **The command line** — a worklist script prints the work queue in the terminal, for the moment of sitting down at the processing machine.

Figure 1, at the front of this paper, is the dashboard's summary layer; Figure 2, in the next section, shows its decision-support core.

12. Decision support: the work queue

The pipeline states exist to answer “what should I work on next?” — so the dashboard's central panel, and the equivalent command-line script, turn them into action lists. There is no to-do list to maintain; every list is a query over the derived states, presented in the order of the pipeline itself:

Action list	Contents (all derived)
Capture more	Targets under their acquisition-goal hours, and planned targets with no sessions yet.
Calibration to shoot	Light-frame combinations whose captured data has no matching dark or bias in the library — the coverage report, below.
Build masters	Bias and dark sets that have raw frames but no master file yet.
To cull	Captured sessions not yet reviewed, with a per-session count of quality-flagged frames (from NINA's embedded focus and guiding figures) so the worst nights sort to the top.
To integrate	Culled sessions not yet stacked.
Restack	Multi-session integrations whose membership rule now matches data that is not in the built master.
To edit	Finished images — single sessions or integrations — that are integrated but not yet edited.

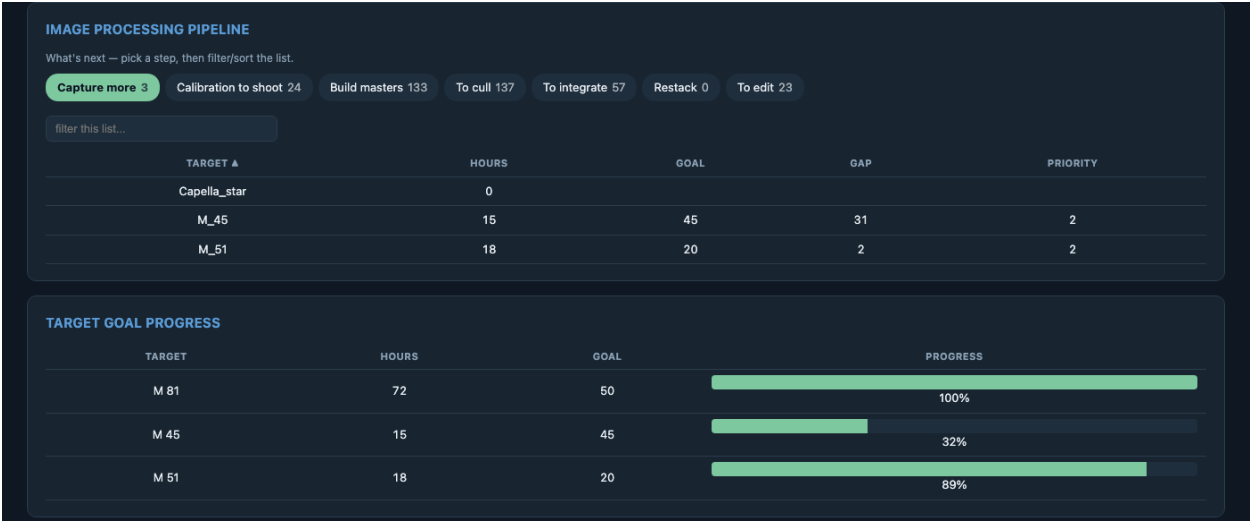


Figure 2 — the work queue: one flat, sortable list per pipeline action, plus acquisition-goal progress. Every row is derived from the files; nothing is maintained by hand.

The newest of these lists deserves a fuller description. The coverage report inverts the usual calibration question: instead of asking what the calibration library contains, it asks whether the data actually captured is calibratable. For every combination of camera, gain, and exposure the kept light frames use, it looks for matching dark data — same camera, gain, and exposure, with a set temperature within five degrees Celsius of the lights' sensor temperature (sets without a temperature folder, such as an uncooled DSLR's, match any) — and matching bias data for the camera. Each combination lands in one of three states: ok (a built master covers every frame), to build (raw calibration frames exist but the master has not been built), or to shoot (some frames have no matching calibration data at all). The report surfaced real gaps immediately: a camera's 120-second lights captured at -10°C with only -20°C darks in the library, and two cameras with no bias or dark data whatsoever.

Because the camera names in session folders and in the calibration library both come from the `sensor_values` registry, this match is an exact string join — a direct payoff of the controlled vocabulary.

13. Presentation conventions

Every reporting face follows a short written style guide, versioned with the code. The rules are small but they compound: the target identifier (M_81) is the working handle everywhere, with the expanded name (“Bodes Galaxy”) shown alongside where space allows, never in place of it. Column headers use full words — abbreviations only when space genuinely demands. Statuses are lower case (ok, to build, stale), and capital letters and red are reserved for act-now urgency such as validation errors — a queue of chores is not an alarm. Exposures drop meaningless trailing decimals (300, not 300.0). Hours round to the nearest whole hour in summaries and totals, and show hours and minutes (2h 30m) for per-item breakdowns; nobody plans around the minutes in a lifetime total. Detail sections in every report follow the pipeline order of Section 8, with summaries first and diagnostic panels last. This paper follows the same conventions.

14. Worked example: M 81 Bodes Galaxy

M 81 is the most heavily imaged target in the combined library: 17 sessions across four telescope-camera combinations and both volumes, totalling 72 hours of kept integration. The folder structure at the M 81 level (excerpted):

```
lifetime archive:
  M 81 Bodes Galaxy/
    M_81 Pleiades111 ASI2600MCAir 2025-01-26
    M_81 Redcat51 ASI585MCPro 2026-02-10
    ...

working volume:
  M 81 Bodes Galaxy/
    M_81 HAC125DX ASI585MCPro 2026-04-18 / 19
```

```

M_81 RASA8 ASI2600MCAir 2026-04-18 / 19 / 20 / 22 / 23
M_81 Redcat51 ASI585MCPro 2026-02-26 / 03-04 / 03-08 / ... / 03-27
integrations/
M_81 RASA8 ASI2600MCAir all/ ← living integration (Section 9)

```

Everything this paper claims can be seen in miniature here. Integration per rig is a grouping query over parsed filenames. The living integration folder accumulates the RASA8 nights under one rule and flags itself stale when a new night lands. The target's 50-hour acquisition goal reads 100 percent complete on the dashboard. And the session keys stay unique even with four rigs on one target, so the tracker re-scans freely.

15. Current library state

The figures below are produced by the tracker from a scan of both libraries, as of July 2026. The most recent full scan finished with zero validation errors and zero warnings.

Metric	Working	Archive	Total
Deep-sky sessions	113	103	216
Kept integration (hours)	377	376	753
Deep-sky targets tracked			75
Targets with kept light frames			72
Kept light frames			10,740
Rejected light frames			1,359
Other-capture sessions			6
Calibration library (bias + dark sets)			133

Top ten deep-sky targets by lifetime integration:

Target	Hours	Sessions	Scopes used
M 81 Bodes Galaxy	72	17	HAC125DX, Pleiades111, RASA8, Redcat51
M 44 Beehive Cluster	50	16	Pleiades111, RASA8, Redcat51
NGC 1499 California Nebula	34	9	Pleiades111, Redcat51
M 106 spiral galaxy	32	9	HAC125DX, Pleiades111, RASA8, Redcat51
M 31 Andromeda Galaxy	30	10	Pleiades111, Redcat51
IC 342 Caldwell 5	28	6	Redcat51
IC 1805 Heart Nebula	24	4	Pleiades111, Redcat51
M 42 Orion Nebula	23	9	Pleiades111, Redcat51

Target	Hours	Sessions	Scopes used
NGC 2403 Caldwell 7	22	6	Pleiades111, Redcat51
M 97 Owl Nebula	21	5	RASA8

Imaging by year: 35 sessions and 124 hours in 2024; 39 sessions and 127 hours in 2025; 142 sessions and 502 hours in 2026 to date.

Appendix A. Controlled vocabularies

A.1 Telescopes and lenses (9)

- HAC125DX
- NexStar6SE
- Pleiades111
- RASA8
- RF100-500
- RF70-200
- Redcat51
- WO50
- ZWO30

HAC125DX, Pleiades111, RASA8, and Redcat51 are the four imaging telescopes that pair with imaging cameras for deep-sky work. NexStar6SE is used for visual and lunar observation; the RF lenses cover landscape and time-lapse; WO50 is a finder; ZWO30 is the guidescope.

A.2 Cameras (10)

- ASI174MM
- ASI220MMmini
- ASI2600MCAir
- ASI432MM
- ASI585MCPro
- Canon5Div
- CanonR5
- PoseidonCPro
- SonyA7Riv
- minicam8

ASI2600MCAir, ASI585MCPro, PoseidonCPro, and minicam8 are the imaging cameras paired with deep-sky telescopes. The monochrome cameras serve narrowband and guiding roles. The Canon and Sony bodies cover landscape, time-lapse, and “other captures”.

A.3 Imaging telescope + camera combinations (17)

- HAC125DX_ASI2600MCAir
- HAC125DX_ASI585MCPro
- HAC125DX_PoseidonCPro
- HAC125DX_minicam8

- Pleiades111_ASI2600MCAir
- Pleiades111_ASI585MCPro
- Pleiades111_PoseidonCPro
- Pleiades111_minicam8
- RASA8_ASI2600MCAir
- RASA8_ASI585MCPro
- RASA8_PoseidonCPro
- RASA8_minicam8
- Redcat51_ASI2600MCAir
- Redcat51_ASI585MCPro
- Redcat51_CanonR5
- Redcat51_PoseidonCPro
- Redcat51_minicam8

A.4 Filters (14)

- B - QHY
- G - QHY
- H3nm - QHY 3nm EVO
- Ha-Hb-Oiii - Celestron RASA8
- IR - ZWO
- L - QHY
- LPI - Light Pollution Imaging - Celestron RASA8
- LQuadE - Optolong L-Quad Enhance Broadband Light Pollution
- LUltDual - Optolong L-Ultimate Dual Bandpass Light Pollution Reduction Imaging
- LeHDual - Optolong L-eNhanche Dual Bandpass Light Pollution Reduction
- O3nm - QHY 3nm EVO
- R - QHY
- S3nm - QHY 3nm EVO
- SA100 - Star Analyser 100

The target vocabulary (81 folders) is omitted for space; it follows the naming convention of Section 5.1.

Appendix B. Onboarding an existing library

The system was not adopted on day one of a new hobby — it was retrofitted onto two years of accumulated captures. Two tools exist specifically to make that safe, and they are the first things to run against any existing library.

B.1 The validation pass

Every ingest ends with a validation pass over the populated database — pure observation, never modifying the library. Each finding carries a severity and a code; the taxonomy covers the ways real libraries actually drift:

Severity	Examples	Meaning
error	EMPTY_SESSION, DATE_MISMATCH, FUTURE_DATE	Something is structurally wrong: a session folder with no frames and no results, or a folder date that contradicts every frame timestamp inside it.
warning	UNKNOWN_SENSOR, UNKNOWN_SCOPE, CAL_UNKNOWN_CAMERA, UNPARSED_SESSION_NAME, UNPARSED_FITS, SENSOR_MISMATCH, MULTI_NIGHT_SPAN	Something does not conform: a name not in the registry (for sessions and for calibration folders alike), a filename no grammar recognizes, a folder sensor that contradicts the FITS header, or frames spanning more nights than the folder claims.
info	NOTES_MISSING, CAL_EMPTY, REGISTRY_ORPHAN	Housekeeping: a session without a notes file, an empty calibration folder, a registry entry never yet imaged.

Findings appear in the dashboard's data-health panel and the workbook. The practical onboarding loop is: run the ingest, read the findings, fix the library (rename, merge, or registry-add), and re-run until the board is clean. Onboarding this library took a handful of such passes: the process surfaced misspelled catalog names, duplicate target folders differing by a typo, truncated camera tokens, and — most consequentially — filename-grammar gaps that had left roughly 3,000 frames uncounted until two additional grammars and filter-name handling were added. The current state is zero errors and zero warnings across both libraries, which is what makes every number in this paper trustworthy.

B.2 The staging preflight

New sessions are staged in a holding folder before being filed into the library. A preflight script checks each staged session: the folder name parses into four valid tokens, the target is known — to the library or to the registry — the rig combination is registered, the frames inside parse, and the move would not collide with an existing session. By default it prints a verdict per session and touches nothing; with an apply flag it files the passing sessions, creating the target folder from the registry when the library does not have it yet. That last detail enforces a deliberate rule: the registry is the single source of truth for target names, and libraries carry no empty target folders — a folder exists in a library only once real data has been filed into it. The same script

resolves the `_adjacent` suffix and reports what a staged piggyback session will attach to. Between the preflight at the front door and the validation pass behind it, a malformed name has no quiet way into the library — which is what keeps a convention alive longer than enthusiasm does.

Appendix C. The nine filename grammars

The parser tries each grammar in order; the first match wins. All matches expose: frame type, exposure value and unit, camera, gain (or ISO), datetime, optional rotation, temperature, optional filter, and a sequence index. The NINA science grammar additionally exposes HFR and RMS. Framing snapshots and preview files are recognized as deliberate non-science and excluded from unparsed-file warnings.

#	Grammar	Shape
1	ASIAir science	<code>Light_{target}_{exp}{s ms}_Bin{N}_{cam}_gain{G}_{dt}_{[rot]deg}_{temp}C_{[filter]}_{idx}.fit</code>
2	ASIAir calibration	As #1: type is Flat/Dark/Bias/DarkFlat, no target token.
3	NINA legacy	<code>LIGHT_{target}_{exp}s_{NxN}_{cam with spaces}_{gain}_{date}_{time}_{rot}_{temp}_{idx}.fi</code>
4	NINA v2 science	<code>LIGHT_{target}_{exp}s_Bin{NxN}_{cam}_gain{G}_{date}_{time}_{[filter]}_{rot}deg_{temp}C__</code>
5	NINA v2 calibration	As #4 with an empty target (FLAT__...).
6	ASIAir DSLR science	<code>Light_{target}_{exp}{s ms}_Bin{N}_ISO{iso}_{dt}_{[rot]deg}_{temp}C_{cam}_{idx}.fit</code>
7	ASIAir DSLR calibration	As #6 without the target token.
8	ASIAir science, rotator-first	<code>Light_{target}_{rot}deg_{exp}{s ms}_Bin{N}_{cam}_gain{G}_{dt}_{temp}C_{idx}.fit</code>
9	ASIAir calibration, rotator-first	As #8 without the target token.

Grammars 8 and 9 cover an early-2026 ASIAir firmware epoch that wrote the rotator angle immediately after the type or target instead of before the temperature. For the DSLR grammars, ISO lands in the gain field (ASIAir's ISO is the DSLR analogue of gain) and the camera token is the free-text setup name typed into the ASIAir. The primary regular expression, for grammar 1:

```
^(?P<type>Light)_(?P<target>.+?)_(?P<exp>[\d.]+)(?P<unit>s|ms)_
Bin(?P<bin>\d+)_(?P<cam>[^_]+)_gain(?P<gain>-\d+)_
(?P<dt>\d{8}-\d{6})_(?: (?P<rot>-\d[\d.]+)deg_)?(?P<temp>-\d[\d.]+)C
(?:_(?P<filter>[A-Za-z][\w ]*))?(?P<idx>\d+)\.(?P<ext>fit|fits|xisf)$
```

The NINA pattern is NINA's default with the literal anchors Bin, gain, deg, and C added so the structure mirrors ASIAir, and a double underscore as a deliberate visual marker between physical metadata and quality-of-capture metadata. HFR, RMS, and the filter may all be empty; the parser accepts either case, including filter names containing spaces.

Appendix D. Representative tracker queries

Three examples, runnable as written against the database. The first computes integration from the frame level; the second and third read the derived views, which is how the dashboard and worklist get their answers.

```
-- Lifetime and this-year integration on a target:
SELECT t.common_name,
  ROUND(SUM(CASE WHEN NOT f.is_rejected THEN f.exp_s END)/3600.0)
  AS hours_lifetime,
  ROUND(SUM(CASE WHEN NOT f.is_rejected
    AND strftime('%Y', f.captured_at_utc)='2026'
    THEN f.exp_s END)/3600.0) AS hours_2026
FROM targets t JOIN sessions s USING (target_id)
JOIN frames f USING (session_id)
WHERE t.target_id='M_81' AND f.frame_type='light' AND f.exp_unit='s'
GROUP BY t.common_name;
```

```
-- Calibration sets needing attention (no master, or stale):
SELECT class, camera, gain, exp_s, status
FROM v_calibration_needs
WHERE status != 'ok' AND class IN ('bias', 'dark')
ORDER BY class, camera, gain, exp_s;
```

```
-- Coverage: captured light combinations with no matching dark data:
SELECT camera, gain, exp_s, light_subs, hours, dark_status, bias_status
FROM v_light_calibration_coverage
WHERE dark_status = 'to shoot'
ORDER BY hours DESC;
```

Appendix E. Other approaches to astrophotography data organization

The structure described in this paper is one point in a wide design space. The approaches below are the ones most discussed in amateur communities such as Cloudy Nights and the AstroBin forums, followed by how two professional observatories handle the same problem. They are offered as a reference for readers weighing the trade-offs.

E.1 Date-first hierarchy

Folders are organized by night first and target second: a dated folder per session, each holding that night's lights and calibration frames. This is the out-of-the-box behavior of NINA, whose default file pattern groups by image type and date, and it mirrors how terrestrial photographers organize shoots. It is the simplest thing to do at capture time — a night's work lands in one place. Its weakness is the question this paper opens with: a single target's data scatters across many date folders, so total integration time per target is tedious to compute and a target's history is never in one view.

E.2 Target-first hierarchy

Folders are organized by target first, with dated session subfolders inside. This is the most frequently recommended structure in amateur forums, on the reasoning that imagers return to the same targets across seasons and years, so the target is the durable unit. Total integration per target is immediate and a target's whole history sits in one folder. It is the family this paper's system belongs to. Its cost is discipline: session subfolders must be named consistently or the advantage erodes — which is exactly the problem a controlled vocabulary and a fixed template are there to solve.

E.3 Capture / Library / Processing separation

The top level is split by how often data changes rather than by target: a Capture tree written every session, a Library tree of master calibration data that changes rarely, and a Processing tree rewritten on every processing run. The appeal is backup strategy — capture data can move to slow archive storage once calibrated, while the small, volatile Processing tree is backed up often. The cost is that one target's material spreads across three trees, so the scheme needs an index to reassemble it. This paper's design adopts the idea in part: calibration lives in its own library, but capture and target data stay unified.

E.4 Frame-type-first within target

Within each target, folders are organized by frame type and then by filter — Light, Dark, Flat, Dark Flat, each subdivided by filter — with rich filenames carrying date, temperature, filter, and exposure. This layout feeds PixInsight's WeightedBatchPreprocessing script with minimal pointing and clicking, since the script consumes frame-type folders directly. It optimizes for the calibration step, at the cost of deeper nesting and a structure organized around a processing tool rather than around the observation.

E.5 Target-plus-attempt

Each target folder contains numbered or dated “attempt” folders, and each attempt is a self-contained mini-project with its own lights, calibration, finals, and processing-application files. This suits imagers who reprocess often and want each reprocessing reproducible in isolation. The weak point is that the boundary of an “attempt” is subjective — when several nights are

combined it is unclear which attempt owns them. This paper's living integrations (Section 9) are the same instinct with the ambiguity removed: membership is a written rule, and what was stacked is a written record.

E.6 Professional practice: Hubble and JWST

The Hubble Space Telescope names every dataset with a nine-character “IPPPSSOOT” rootname that encodes the instrument, proposal, observation set, and observation in fixed-width positions; each file is the rootname plus a suffix indicating its processing level — raw, calibrated, drizzled, and so on. JWST uses a comparable scheme of program and observation identifiers. In both cases the filenames are deliberately machine-oriented — the documentation states plainly that they are not designed to be scientist-friendly — and all human searching is done through the Mikulski Archive for Space Telescopes, a relational archive that indexes the opaque filenames by target name, program, instrument, exposure type, and date.

The professional lesson is the separation of two concerns: a rigid, machine-parseable identity encoded in the filename, and a separate searchable layer — a database — built on top of it. This paper's system follows the same division: self-describing folder and file names provide the rigid identity, and the SQLite tracker provides the searchable layer. The deliberate difference is one of scale. An amateur archive of under a hundred targets can afford folder names that are human-readable as well as machine-parseable, so the imager gets both and rarely needs the database for routine work. The database earns its place for the aggregate questions — integration totals, calibration coverage, pipeline state — that no single folder name can answer.

Availability and acknowledgements

The toolchain is plain Python and SQLite — no services, no daemons. The only third-party dependency is the library used to write the Excel workbook, and the only network call is one request per session to a free historical-weather service (Open-Meteo) to fill the notes file's weather section. The repository (scripts, schema, configuration template, and the style guide) will be shared publicly so the system can be run rather than just read; the capture data itself, of course, stays home.

This revision describes a working system, not a proposal — every number in it was produced by the code it documents. Comments, corrections, and counter-arguments from other astrophotographers remain welcome; several decisions in this revision (the culled stage, the ledger model, dropping machine tracking) came directly from using the first draft's design and finding its edges.

One ask in particular: we would love to hear of other examples of astrophotography library management — formal or improvised, at any scale — that can help strengthen this design in its next iteration.

Thanks to anyone that read this far,

Steve